

Linked Lists: FAANG / MAANG Revision Guide

A pattern-first study guide for linked-list interviews and revision.

A practical revision guide for the linked-list questions covered in `linkedList_into.py` and `linkedList_advanced.py`.

1. Mental Model

A linked list is a chain of node objects. Each node stores data and a link to another node.

```
head -> [data | next] -> [data | next] -> None
```

The list is controlled by head. There is no random indexing: reaching position i costs $O(i)$. The most important skill is changing links without losing the rest of the chain.

Three rules

1. Save the next node before changing `current.next`.
2. Use a dummy node when the head may be inserted, removed, or replaced.
3. Compare node identity (`is`), not only node values, for cycles and intersections.

Complexity baseline

Operation	Time	Extra space
Traverse/search	$O(n)$	$O(1)$
Insert at head	$O(1)$	$O(1)$
Insert at tail without tail pointer	$O(n)$	$O(1)$
Delete with previous node	$O(1)$	$O(1)$
Reverse in place	$O(n)$	$O(1)$

2. Pointer Patterns To Memorize

Pattern A: Previous, current, next

Use for reversal. The `next_node` variable prevents the remaining list from being lost.

```
previous_node = None
current_node = head
while current_node is not None:
    next_node = current_node.next
    current_node.next = previous_node
    previous_node = current_node
    current_node = next_node
head = previous_node
```

Memory cue: **save, flip, advance**.

Pattern B: Slow and fast pointers

Use for middle, cycle detection, palindrome, and splitting.

```
slow = head
fast = head
while fast and fast.next:
    slow = slow.next
    fast = fast.next.next
```

Memory cue: **one walks, one runs.**

Pattern C: Two pointers with a gap

Use for kth-from-end and remove nth-from-end. Move fast ahead first, then move both together.

For deletion with a dummy node, create a gap of $n + 1$ so slow stops immediately before the target.

Memory cue: **keep the gap; the end tells the answer.**

Pattern D: Dummy node

```
dummy_node = Node(0)
dummy_node.next = head
```

Return `dummy_node.next`. This removes special cases for deleting or inserting at the head.

Memory cue: **dummy protects the head.**

Pattern E: Split, reverse, weave

Used by palindrome and reorder:

1. Find the middle.
2. Reverse the second half.
3. Compare or interleave the halves.

Memory cue: **cut, turn, combine.**

Pattern F: Hash map plus linked structure

Use a map for $O(1)$ lookup and links for order. This is the core of random-pointer copying and LRU/LFU caches.

3. Foundational Problems

Create, insert, print, delete

A Node stores data and next. A list stores head.

Insertion at the end walks until `current.next` is None, then assigns the new node. Printing walks until None.

To delete a value, inspect `current.next`; then bypass it:

```
current.next = current.next.next
```

Always handle: empty list, deleting head, deleting tail, missing value, and duplicate values. The implementation removes the first matching value.

Reverse Linked List

Use previous/current/next. The answer is the final `previous_node`.

- Time: $O(n)$
- Space: $O(1)$

Reverse recursively

Base case: empty list or one node. Recursively reverse the suffix, then place the current node after its next node:

```
new_head = reverse(current.next)
current.next.next = current
current.next = None
return new_head
```

- Time: $O(n)$
- Call-stack space: $O(n)$

Middle of the list

Slow moves one node; fast moves two. With the usual loop, even-length lists return the second middle.

- Time: $O(n)$

- Space: $O(1)$

kth node from the end

Advance fast by k, then move both pointers until fast is None. The slow pointer is the answer.

- Time: $O(n)$
- Space: $O(1)$

4. Cycle Problems

Linked List Cycle

Floyd's algorithm: if slow and fast ever refer to the same object, a cycle exists.

```
if slow_node is fast_node:
    return True
```

Do not compare only data; different nodes may have equal data.

- Time: $O(n)$
- Space: $O(1)$

Linked List Cycle II

After slow and fast meet, reset a pointer to head. Move both one step. Their next meeting point is the cycle entrance.

Why it works: the distance from head to cycle entrance equals the appropriate distance from the meeting point around the cycle.

- Time: $O(n)$
- Space: $O(1)$

Cycle length

After finding a meeting point, keep one pointer fixed and walk the other until it returns. Count steps.

5. Two Lists and Merging

Merge two sorted lists

Use a dummy node. Attach the smaller current node, advance that list, and continue. Finally attach the remaining suffix.

- Time: $O(n + m)$
- Space: $O(1)$

Merge k sorted lists

Put every non-empty head into a min-heap. Pop the smallest node and push its next node.

Heap entries should include a tie-breaker such as list index because Python cannot always compare node objects.

- Time: $O(N \log k)$
- Space: $O(k)$

Intersection of two linked lists

Intersection means the same node object, not equal data. Let each pointer switch to the other head when it reaches None. Both then travel equal total distance.

- Time: $O(n + m)$
- Space: $O(1)$

6. Deletion and Arrangement

Remove nth node from end

Use a dummy node and a gap of $n + 1$. When fast reaches None, slow is before the target:

```
slow.next = slow.next.next
```

- Time: $O(n)$
- Space: $O(1)$

Remove duplicates from sorted list

Equal values are adjacent. If `current.data == current.next.data`, bypass the next node; otherwise advance.

- Time: $O(n)$
- Space: $O(1)$

Remove duplicates from unsorted list

Track seen values in a set. If a value is repeated, bypass the current node.

- Time: $O(n)$ average
- Space: $O(n)$

Reverse a sublist

Use a dummy node and locate the node before `left`. Repeatedly take the next node and move it to the front of the sublist.

- Time: $O(n)$
- Space: $O(1)$

Swap nodes in pairs

For each pair, reconnect `previous -> second -> first -> rest`. Leave an odd final node unchanged.

- Time: $O(n)$
- Space: $O(1)$

Rotate right by k

Find length and tail, make the list temporarily circular, then break it at position `length - k`.

Always reduce first: `k %= length`.

- Time: $O(n)$
- Space: $O(1)$

Reorder list

For `1 -> 2 -> 3 -> 4 -> 5`, produce `1 -> 5 -> 2 -> 4 -> 3`.

Recipe: find middle, reverse second half, weave alternating nodes.

- Time: $O(n)$
- Space: $O(1)$

7. Palindrome and Number Problems

Palindrome linked list

Split at the middle, reverse the second half, compare values, then restore the second half if preserving input matters.

- Time: $O(n)$
- Space: $O(1)$

Add two numbers

Digits are usually stored in reverse order. Add matching digits plus carry; create a result node for `total % 10`, and carry `total // 10`.

Keep looping while either list or carry remains.

- Time: $O(\max(n, m))$
- Result space: $O(\max(n, m))$

Add one

For forward-order digits, recurse to the tail and propagate carry backward. If the final carry remains, create a new head.

Example: 1 -> 2 -> 9 becomes 1 -> 3 -> 0.

8. Special Node Structures

Copy list with random pointer, hash-map version

Make one copy per original node, map original to copy, then assign next and random using the map. This is simple and $O(n)$ extra space.

Copy random list with $O(1)$ extra space

Three passes:

1. Insert each copy after its original: A -> A' -> B -> B'.
2. Set `copy.random = original.random.next`.
3. Separate original and copied chains.

Memory cue: **interleave, wire, separate**.

- Time: $O(n)$
- Extra space: $O(1)$

Flatten a multilevel doubly linked list

When a node has a child, recursively flatten the child, splice it between the node and its former next, repair both previous links, and continue.

Always set `child = None` after splicing.

9. Sorting

Merge sort on a linked list

Find the midpoint with slow/fast, split the list, recursively sort both halves, and merge them.

Linked lists prefer merge sort because splitting and merging use links and no random access.

- Time: $O(n \log n)$
- Call-stack space: $O(\log n)$

10. Queue, Deque, and Caches

Queue with linked list

Keep both head and tail:

- enqueue at tail: $O(1)$
- dequeue at head: $O(1)$

When dequeue empties the list, set both pointers to None.

Deque with doubly linked list

A doubly linked list supports $O(1)$ insertion and removal at both ends. Keep head and tail, and repair neighboring links on every operation.

LRU cache

Use a dictionary plus doubly linked list. The front is most recently used; the back is least recently used.

- get: find in map, remove, move to front
- put: update or add at front
- over capacity: remove node before tail and delete its map entry

Both operations: $O(1)$.

LFU cache

Track each key's node and frequency buckets. Each bucket preserves recency with an ordered dictionary. Evict from the lowest frequency; break ties by least recent.

The key state is `minimum_frequency`.

- Average get/put: $O(1)$
- Space: $O(\text{capacity})$

11. Josephus Problem

For n people in a circle, remove every k th person until one remains. A circular linked list models the process directly: maintain previous and current, advance $k - 1$, bypass current, and continue.

- Time: $O(nk)$ with direct simulation
- Space: $O(n)$

For interviews, also know the recurrence for the zero-based survivor:

$$\begin{aligned} J(1, k) &= 0 \\ J(n, k) &= (J(n - 1, k) + k) \bmod n \end{aligned}$$

Convert back to one-based numbering by adding 1.

12. Edge-Case Checklist

Before coding, ask:

- Is the list empty?
- Is there one node?
- Does the answer remove or replace the head?
- Is the answer at the tail?
- Are there duplicate values?
- Should comparison use value or node identity?
- Can k or n be zero, negative, or larger than the length?
- Is the input allowed to be modified?
- Could a loop become infinite because the list is cyclic?
- Does a doubly linked list need both directions repaired?
- Does a cache update an existing key's recency/frequency?

13. Interview Answer Template

1. Restate the input and expected output.
2. State the invariant: what does each pointer mean?
3. Name the pattern: dummy, slow/fast, gap, reversal, heap, or map plus links.
4. Walk through a small example.
5. Code while preserving the invariant.

6. Test empty, one-node, head, tail, duplicate, and invalid cases.
7. State time and space complexity.

14. Memory Map

- Reverse: **save, flip, advance**
- Middle/cycle: **one walks, one runs**
- kth from end: **keep a gap**
- Delete head safely: **dummy protects head**
- Palindrome/reorder: **cut, turn, combine**
- Random copy $O(1)$: **interleave, wire, separate**
- Merge k: **heap of current heads**
- LRU: **map finds, list orders**
- LFU: **frequency buckets plus recency**
- Queue: **head out, tail in**
- Deque: **two-sided doubly linked list**
- Josephus: **circular list, bypass kth**

15. Revision Plan

Pass 1: Fundamentals

Implement Node, insert, traverse, delete, reverse, middle, kth from end, and cycle detection from memory.

Pass 2: Pointer combinations

Solve cycle start, remove nth from end, merge two lists, intersection, palindrome, and reorder.

Pass 3: Transformations

Solve recursive reverse, duplicate removal, sublist reversal, pair swap, rotation, and merge sort.

Pass 4: Specialized structures

Solve random-pointer copy, multilevel flattening, queue, deque, LRU, LFU, and Josephus.

Spaced repetition

- Same day: explain each invariant aloud.
- Next day: write the core template without looking.
- Three days later: solve one edge-case-heavy variant.
- One week later: complete a timed mixed set.

Final Target

You are interview-ready when you can identify the pattern within one minute, explain the pointer invariant before coding, handle head/tail/empty cases, and give the correct complexity without relying on a memorized implementation.